



Regret, Exploration, and Exploitation

The Language of Bandit Theory

Author: Tang

Institute: Tang's Machine Learning Blog

Date: 2026-06-17

Version: 1.0.0

Topic: Bandits, Regret, Exploration, Sequential Decision-Making

Contents

Chapter 1 The first word is regret	1
1.1 The tiny laboratory	1
1.2 Regret as one-line bookkeeping	2
Chapter 2 Exploration and exploitation are not slogans	4
2.1 The two costs in one toy calculation	4
Chapter 3 The algorithms in a small experiment	6
3.1 Greedy	6
3.2 ϵ -greedy	6
3.3 UCB	6
3.4 Thompson sampling	6
3.5 The code that creates the comparison	7
3.6 What the experiment says	7
Chapter 4 A little probability, used as a ruler	9
4.1 Why the logarithm appears	9
Chapter 5 UCB proof, slowly	11
5.1 The good event	11
5.2 What must be true if UCB pulls a bad arm	11
5.3 Turning pull count into regret	12
5.4 Why small gaps are different	12
Chapter 6 Thompson sampling as probability matching	14
Chapter 7 What this language buys us	15
Appendix A. Symbol Table	16
Appendix B. Full Experiment Script	17
References	20

Chapter 1 The first word is regret

A bandit problem begins with a small annoyance.

You choose an action. You see the reward of that action. The rewards of the actions you did not choose disappear.

This is not a technical detail. It is the whole problem. In supervised learning, a training example usually tells us the correct label. In a bandit problem, one action gives one receipt. A receipt is not an answer key.

The word *regret* is the cleanest way to measure the cost of this missing answer key.

Key idea

Regret is not sadness. It is an accounting identity. It asks: after T decisions, how much reward did we lose compared with always taking the best action?

Imagine three possible headlines for the same article. The true click probabilities are

$$0.30, \quad 0.35, \quad 0.42.$$

The third headline is best, but the learner does not know this. If it shows the first headline, it observes only whether the first headline was clicked. It does not observe whether the second or third headline would have been clicked by the same reader.

That is the whole difficulty:

we need data to choose well, but choosing badly is how we get some of the data.

1.1 The tiny laboratory

We use a K -armed Bernoulli bandit. There are K actions, called arms. Pulling arm a gives a random reward

$$X_{a,t} \in \{0, 1\},$$

with mean

$$\mathbb{E}[X_{a,t}] = \mu_a.$$

For the running example,

$$(\mu_1, \mu_2, \mu_3) = (0.30, 0.35, 0.42).$$

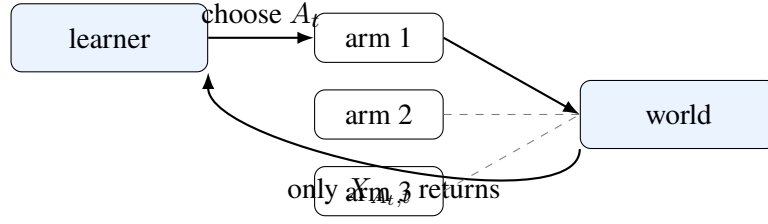
The best mean is

$$\mu_\star = \max_{a \in \{1, \dots, K\}} \mu_a = 0.42.$$

At round t , the learner chooses an arm A_t and observes only

$$X_{A_t,t}.$$

It does not observe $X_{a,t}$ for $a \neq A_t$.



The dashed arrows are the counterfactual rewards. They exist in our mathematical imagination, but they are not observed by the learner.

1.2 Regret as one-line bookkeeping

Suppose the learner chooses arms

$$A_1, A_2, \dots, A_T.$$

The oracle, who knows the best arm, receives expected reward

$$T\mu_\star.$$

The learner receives expected reward

$$\mathbb{E} \left[\sum_{t=1}^T X_{A_t, t} \right].$$

So the expected regret is

$$R_T = T\mu_\star - \mathbb{E} \left[\sum_{t=1}^T X_{A_t, t} \right].$$

Now write it in the useful form. The reward of the chosen arm has conditional mean μ_{A_t} . Therefore

$$\begin{aligned}
 R_T &= T\mu_\star - \mathbb{E} \left[\sum_{t=1}^T X_{A_t, t} \right] \\
 &= \mathbb{E} \left[\sum_{t=1}^T \mu_\star \right] - \mathbb{E} \left[\sum_{t=1}^T X_{A_t, t} \right] \\
 &= \mathbb{E} \left[\sum_{t=1}^T (\mu_\star - X_{A_t, t}) \right] \\
 &= \sum_{t=1}^T \mathbb{E} [\mu_\star - X_{A_t, t}].
 \end{aligned}$$

The only probabilistic step is the next one:

$$\mathbb{E}[X_{A_t, t} \mid A_t] = \mu_{A_t}.$$

Hence

$$\begin{aligned}
 \mathbb{E} [\mu_\star - X_{A_t, t}] &= \mathbb{E} [\mathbb{E} [\mu_\star - X_{A_t, t} \mid A_t]] \\
 &= \mathbb{E} [\mu_\star - \mathbb{E}[X_{A_t, t} \mid A_t]] \\
 &= \mathbb{E} [\mu_\star - \mu_{A_t}].
 \end{aligned}$$

Therefore

$$R_T = \mathbb{E} \left[\sum_{t=1}^T (\mu_{\star} - \mu_{A_t}) \right].$$

This is the basic accounting formula. Define the gap of arm a by

$$\Delta_a = \mu_{\star} - \mu_a.$$

If arm a is pulled $N_a(T)$ times, then

$$\begin{aligned} R_T &= \mathbb{E} \left[\sum_{t=1}^T \Delta_{A_t} \right] \\ &= \mathbb{E} \left[\sum_{t=1}^T \sum_{a=1}^K \Delta_a \mathbf{1}\{A_t = a\} \right] \\ &= \sum_{a=1}^K \Delta_a \mathbb{E} \left[\sum_{t=1}^T \mathbf{1}\{A_t = a\} \right] \\ &= \sum_{a=1}^K \Delta_a \mathbb{E}[N_a(T)]. \end{aligned}$$

Think

This formula is the first real language of bandit theory. To control regret, we do not need to predict every reward. We need to control how often the algorithm pulls arms with positive gap.

Chapter 2 Exploration and exploitation are not slogans

People often describe bandits by saying “exploration versus exploitation.” The phrase is correct, but too vague. Regret makes it precise.

If we pull a bad arm, we lose reward now. That is exploration cost.

If we avoid an uncertain arm too early, we may never learn that it is good. That is exploitation cost caused by premature certainty.

The real question is:

How many times should uncertainty be allowed to buy more information?

2.1 The two costs in one toy calculation

Consider two arms with

$$\mu_1 < \mu_2, \quad \Delta = \mu_2 - \mu_1 > 0.$$

A simple strategy is *explore-then-commit*:

1. Pull each arm m times.
2. Compare their sample means.
3. Use the arm with larger sample mean for the remaining $T - 2m$ rounds.

If the algorithm explores arm 1 exactly m times, that part alone costs

$$m\Delta.$$

The more dangerous event is choosing arm 1 after exploration. Let

$$\hat{\mu}_{1,m} = \frac{1}{m} \sum_{i=1}^m X_{1,i}, \quad \hat{\mu}_{2,m} = \frac{1}{m} \sum_{i=1}^m X_{2,i}.$$

The algorithm commits to the wrong arm when

$$\hat{\mu}_{1,m} \geq \hat{\mu}_{2,m}.$$

Since $\mu_2 - \mu_1 = \Delta$, this event implies at least one of the following two things:

$$\hat{\mu}_{1,m} - \mu_1 \geq \frac{\Delta}{2}, \quad \mu_2 - \hat{\mu}_{2,m} \geq \frac{\Delta}{2}.$$

Indeed, if neither happened, then

$$\hat{\mu}_{1,m} < \mu_1 + \frac{\Delta}{2} = \frac{\mu_1 + \mu_2}{2} = \mu_2 - \frac{\Delta}{2} < \hat{\mu}_{2,m},$$

so the wrong commitment would be impossible.

Therefore, by the union bound,

$$\mathbb{P}(\hat{\mu}_{1,m} \geq \hat{\mu}_{2,m}) \leq \mathbb{P}\left(\hat{\mu}_{1,m} - \mu_1 \geq \frac{\Delta}{2}\right) + \mathbb{P}\left(\mu_2 - \hat{\mu}_{2,m} \geq \frac{\Delta}{2}\right).$$

For rewards in $[0, 1]$, Hoeffding's inequality says

$$\mathbb{P}(\hat{\mu}_{a,m} - \mu_a \geq \varepsilon) \leq \exp(-2m\varepsilon^2).$$

Use $\varepsilon = \Delta/2$:

$$\begin{aligned}\mathbb{P}(\hat{\mu}_{1,m} \geq \hat{\mu}_{2,m}) &\leq \exp\left(-2m\frac{\Delta^2}{4}\right) + \exp\left(-2m\frac{\Delta^2}{4}\right) \\ &= 2\exp\left(-\frac{m\Delta^2}{2}\right).\end{aligned}$$

Hence the expected regret of explore-then-commit is bounded by

$$R_T \leq m\Delta + (T - 2m)\Delta \cdot 2\exp\left(-\frac{m\Delta^2}{2}\right).$$

This is the tradeoff in its simplest form:

$$\text{explore more} \implies m\Delta \text{ grows,}$$

but

$$\text{explore more} \implies 2e^{-m\Delta^2/2} \text{ shrinks.}$$

Key idea

Exploration is not curiosity. It is an insurance premium against choosing the wrong arm for a long time.

Chapter 3 The algorithms in a small experiment

We now run a small experiment. The environment has three Bernoulli arms:

$$(0.30, 0.35, 0.42).$$

The horizon is $T = 2000$, and we repeat the experiment over 500 independent runs.

We compare four algorithms.

3.1 Greedy

Greedy uses the current sample mean and always chooses the arm that looks best:

$$A_t \in \arg \max_a \hat{\mu}_a(t).$$

It is natural, but fragile. A lucky first click can make a bad arm look good. Once that happens, greedy may keep feeding the same mistaken belief.

3.2 ε -greedy

ε -greedy mostly behaves greedily, but sometimes chooses a random arm:

$$A_t = \begin{cases} \text{a uniformly random arm,} & \text{with probability } \varepsilon, \\ \arg \max_a \hat{\mu}_a(t), & \text{with probability } 1 - \varepsilon. \end{cases}$$

This is the simplest repair. It forces the algorithm to keep looking around.

3.3 UCB

UCB chooses the arm with the largest upper confidence index:

$$A_t \in \arg \max_a \left\{ \hat{\mu}_a(t) + \sqrt{\frac{2 \log t}{N_a(t)}} \right\}.$$

The first term is what we have seen. The second term is how uncertain we still are. An arm can be chosen either because it has performed well or because it has not been tested enough.

3.4 Thompson sampling

For Bernoulli rewards, Thompson sampling keeps a Beta posterior for each arm:

$$\theta_a \sim \text{Beta}(\alpha_a, \beta_a).$$

At each round it samples one possible world from the posterior and acts greedily in that sampled world:

$$A_t \in \arg \max_a \theta_a.$$

A click updates

$$\alpha_a \leftarrow \alpha_a + 1,$$

and a non-click updates

$$\beta_a \leftarrow \beta_a + 1.$$

3.5 The code that creates the comparison

The important part is the action rule. The environment is intentionally small, so that the algorithmic difference is not hidden behind engineering.

Core action rules for the experiment.

```

1  if policy == "Greedy":
2      means = sums / np.maximum(counts, 1)
3      a = int(np.argmax(means))
4
5  elif policy == "Epsilon-greedy":
6      if rng.random() < epsilon:
7          a = int(rng.integers(K))
8      else:
9          means = sums / np.maximum(counts, 1)
10         a = int(np.argmax(means))
11
12 elif policy == "UCB":
13     means = sums / counts
14     bonus = np.sqrt(2.0 * np.log(max(t + 1, 2)) / counts)
15     a = int(np.argmax(means + bonus))
16
17 elif policy == "Thompson sampling":
18     theta = rng.beta(alpha, beta)
19     a = int(np.argmax(theta))

```

3.6 What the experiment says

Table 3.1: Results over 500 runs, horizon $T = 2000$.

Algorithm	Final regret	Arm 1 pulls	Arm 2 pulls	Arm 3 pulls
Greedy	137.51	932.35	366.19	701.46
ϵ -greedy	39.85	140.61	328.29	1531.10
UCB	67.29	286.26	470.60	1243.14
Thompson sampling	27.77	98.75	227.37	1673.88

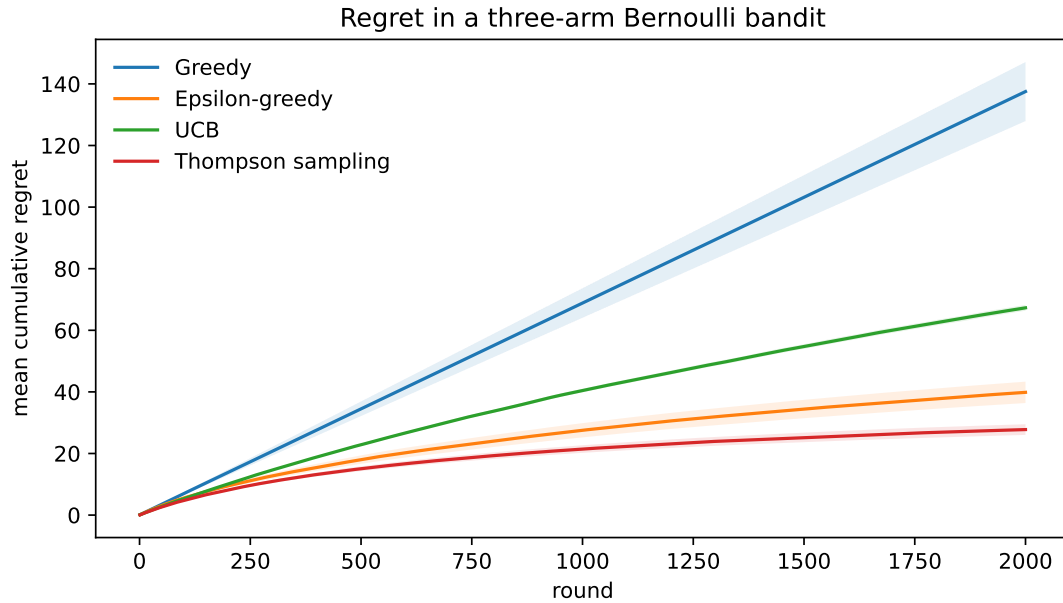


Figure 3.1: Mean cumulative regret. The bands show approximately two standard errors.

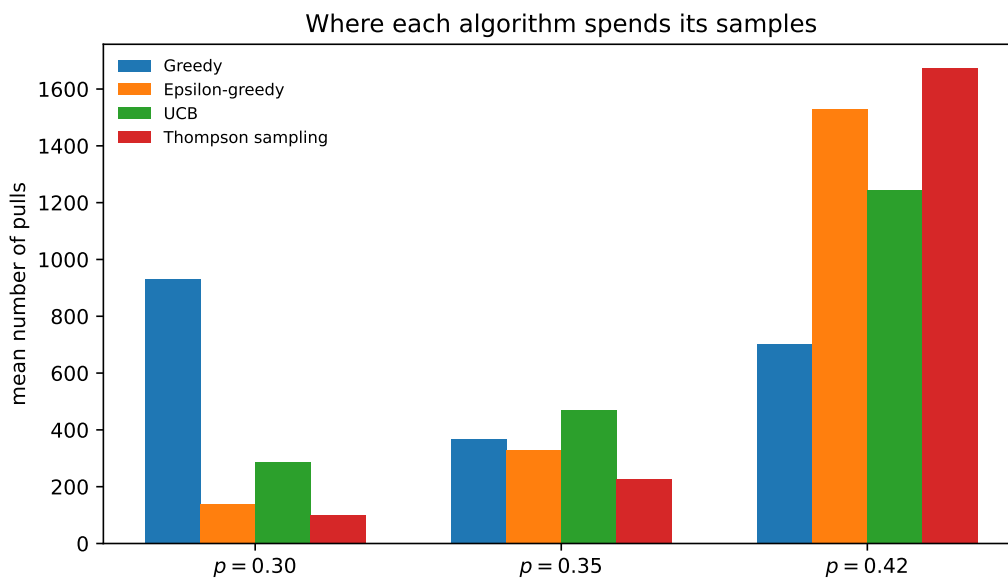


Figure 3.2: Mean number of pulls of each arm. The best arm has mean 0.42.

The greedy algorithm is not bad because it is stupid. It is bad because it has no mechanism for doubting its early evidence.

UCB and Thompson sampling are different ways of putting doubt into the action rule. UCB adds an explicit error bar. Thompson sampling randomizes according to posterior uncertainty. Both are saying: an arm that has not been tested enough should not be declared dead too quickly.

Chapter 4 A little probability, used as a ruler

The probability needed for this note is modest. We use it as a ruler for sample averages.

If arm a has mean μ_a , then after n pulls its sample mean is

$$\hat{\mu}_{a,n} = \frac{1}{n} \sum_{i=1}^n X_{a,i}.$$

This is a noisy ruler. It points near μ_a , but not exactly at μ_a .

Hoeffding's inequality says that for rewards in $[0, 1]$,

$$\mathbb{P}(|\hat{\mu}_{a,n} - \mu_a| \geq r) \leq 2 \exp(-2nr^2).$$

If we want the error probability to be at most δ , solve

$$2 \exp(-2nr^2) = \delta.$$

Step by step:

$$\begin{aligned} 2 \exp(-2nr^2) &= \delta, \\ \exp(-2nr^2) &= \frac{\delta}{2}, \\ -2nr^2 &= \log \frac{\delta}{2}, \\ 2nr^2 &= \log \frac{2}{\delta}, \\ r^2 &= \frac{\log(2/\delta)}{2n}, \\ r &= \sqrt{\frac{\log(2/\delta)}{2n}}. \end{aligned}$$

Thus a natural error bar is

$$\hat{\mu}_{a,n} \pm \sqrt{\frac{\log(2/\delta)}{2n}}.$$

Think

A confidence interval is not a magical theorem. It is just a ruler with a warning label: after n samples, the ruler is wrong by more than r with probability at most δ .

4.1 Why the logarithm appears

If we check one arm once, a small failure probability is enough. But a bandit algorithm checks many arms at many times. We want all of the rulers to be correct at once.

Suppose there are K arms and T rounds. There are at most KT pairs (a, n) to worry about. If each pair fails with probability at most $2/T^4$, then the probability that any pair fails is at most

$$KT \cdot \frac{2}{T^4} = \frac{2K}{T^3}.$$

This is the union bound. It says:

$$\mathbb{P}(\text{at least one bad event}) \leq \sum \mathbb{P}(\text{each bad event}).$$

Now choose the radius

$$r_{a,n} = \sqrt{\frac{2 \log T}{n}}.$$

Then

$$\begin{aligned} \mathbb{P} \left(|\hat{\mu}_{a,n} - \mu_a| \geq \sqrt{\frac{2 \log T}{n}} \right) &\leq 2 \exp \left(-2n \cdot \frac{2 \log T}{n} \right) \\ &= 2 \exp(-4 \log T) \\ &= \frac{2}{T^4}. \end{aligned}$$

So with high probability, for every arm and every sample size,

$$|\hat{\mu}_{a,n} - \mu_a| \leq \sqrt{\frac{2 \log T}{n}}.$$

This is the mathematical reason for the UCB bonus.

Chapter 5 UCB proof, slowly

We now prove the standard idea behind UCB regret. The proof is not long. The hard part is knowing what each line is trying to say.

Let

$$\text{rad}_a(t) = \sqrt{\frac{2 \log T}{N_a(t)}}.$$

UCB uses the index

$$\text{UCB}_a(t) = \hat{\mu}_a(t) + \text{rad}_a(t).$$

At time t , it chooses an arm with largest index.

5.1 The good event

Define the good event

$$\mathcal{G} = \left\{ \forall a, \forall n \leq T : |\hat{\mu}_{a,n} - \mu_a| \leq \sqrt{\frac{2 \log T}{n}} \right\}.$$

From the previous calculation,

$$\mathbb{P}(\mathcal{G}^c) \leq \frac{2K}{T^3}.$$

On $\mathcal{G}$, every empirical mean is close to its truth at every time. So the rest of the proof is deterministic bookkeeping.

5.2 What must be true if UCB pulls a bad arm

Fix a suboptimal arm a , so

$$\Delta_a = \mu_\star - \mu_a > 0.$$

Suppose UCB pulls arm a at some time t . Since UCB chooses the largest index,

$$\hat{\mu}_a(t) + \text{rad}_a(t) \geq \hat{\mu}_\star(t) + \text{rad}_\star(t).$$

On the good event,

$$\hat{\mu}_\star(t) + \text{rad}_\star(t) \geq \mu_\star.$$

Therefore,

$$\hat{\mu}_a(t) + \text{rad}_a(t) \geq \mu_\star.$$

Again on the good event,

$$\hat{\mu}_a(t) \leq \mu_a + \text{rad}_a(t).$$

Combine the two inequalities:

$$\begin{aligned} \mu_\star &\leq \hat{\mu}_a(t) + \text{rad}_a(t) \\ &\leq \mu_a + \text{rad}_a(t) + \text{rad}_a(t) \\ &= \mu_a + 2\text{rad}_a(t). \end{aligned}$$

Thus

$$\Delta_a = \mu_\star - \mu_a \leq 2\text{rad}_a(t).$$

Substitute the radius:

$$\Delta_a \leq 2\sqrt{\frac{2\log T}{N_a(t)}}.$$

Square both sides:

$$\Delta_a^2 \leq 4 \cdot \frac{2\log T}{N_a(t)} = \frac{8\log T}{N_a(t)}.$$

Rearrange:

$$N_a(t) \leq \frac{8\log T}{\Delta_a^2}.$$

Key idea

This is the whole UCB proof in one sentence: once a bad arm has been pulled enough, its uncertainty bonus becomes too small to hide its gap.

5.3 Turning pull count into regret

If arm a is suboptimal, then on the good event it can be pulled at most about

$$\frac{8\log T}{\Delta_a^2}$$

times. Each pull costs Δ_a . Therefore its regret contribution is at most

$$\Delta_a \cdot \frac{8\log T}{\Delta_a^2} = \frac{8\log T}{\Delta_a}.$$

Summing over suboptimal arms,

$$R_T \lesssim \sum_{a:\Delta_a>0} \frac{8\log T}{\Delta_a}.$$

This is called a gap-dependent bound. It is strong when the gaps are not too small.

5.4 Why small gaps are different

If Δ_a is tiny, pulling arm a is not very harmful. The previous bound becomes large because it tries to identify a tiny difference very accurately. But a tiny difference does not need to be identified quickly, because its regret cost is small.

A simple split captures this. Choose a threshold $\varepsilon > 0$.

For arms with small gap $\Delta_a \leq \varepsilon$, the regret is at most

$$T\varepsilon.$$

For arms with larger gap $\Delta_a > \varepsilon$, use the UCB count:

$$\sum_{\Delta_a>\varepsilon} \frac{8\log T}{\Delta_a} \leq \sum_{\Delta_a>\varepsilon} \frac{8\log T}{\varepsilon} \leq \frac{8K\log T}{\varepsilon}.$$

Thus

$$R_T \lesssim T\varepsilon + \frac{8K\log T}{\varepsilon}.$$

Balance the two terms:

$$T\varepsilon = \frac{8K \log T}{\varepsilon}.$$

Then

$$\varepsilon^2 = \frac{8K \log T}{T}, \quad \varepsilon = \sqrt{\frac{8K \log T}{T}}.$$

Substitute back:

$$R_T \lesssim \sqrt{KT \log T}.$$

This is a gap-independent statement. It says that even if the gaps are tiny, regret grows much slower than T .

Chapter 6 Thompson sampling as probability matching

UCB says: use an optimistic upper bound. Thompson sampling says: sample a possible world and act as if that world were true.

For Bernoulli rewards, start with

$$\theta_a \sim \text{Beta}(1, 1).$$

After S_a successes and F_a failures,

$$\theta_a \mid \text{data} \sim \text{Beta}(1 + S_a, 1 + F_a).$$

At each round:

$$\theta_1, \dots, \theta_K \text{ are sampled independently from their posteriors,}$$

and the learner chooses

$$A_t \in \arg \max_a \theta_a.$$

Think

Thompson sampling does not add a bonus. It lets uncertainty itself randomize the decision. An uncertain arm still has a chance to look best in a plausible sampled world.

This is why Thompson sampling often feels less mechanical than UCB. It does not ask each arm for an explicit optimism certificate. It asks the posterior: how often is this arm the winner in worlds that still look plausible?

In symbols,

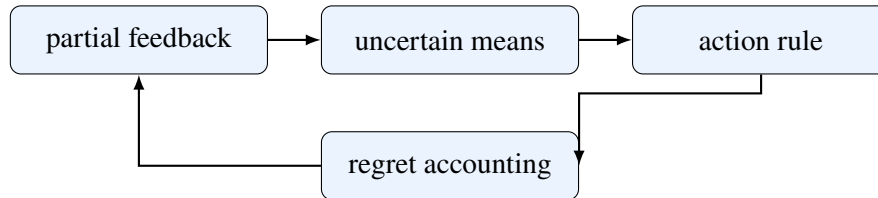
$$\mathbb{P}(A_t = a \mid \mathcal{H}_{t-1}) = \mathbb{P}\left(\theta_a = \max_j \theta_j \mid \mathcal{H}_{t-1}\right),$$

where $\mathcal{H}_{t-1}$ is the data observed before round t .

This is sometimes called probability matching. The probability of choosing an arm is matched to the posterior probability that the arm is optimal.

Chapter 7 What this language buys us

The point of this note is not that UCB and Thompson sampling are the only algorithms worth knowing. The point is that they teach the basic grammar of sequential learning.



Once you see this grammar, many later topics become less mysterious:

- Linear bandits replace arm means by a parameterized reward model.
- Contextual bandits let the best action depend on observed context.
- Bayesian optimization uses posterior uncertainty over functions.
- Best-arm identification changes the objective from earning reward to finding the best arm.
- Reinforcement learning adds state, delayed reward, and credit assignment.

The same question keeps returning: what should an algorithm do with uncertainty when every experiment has a cost?

Appendix A. Symbol Table

Symbol	Meaning
K	number of arms
T	time horizon
A_t	action chosen at round t
$X_{a,t}$	reward of arm a at round t
μ_a	mean reward of arm a
$\mu_\star$	best mean reward
Δ_a	gap $\mu_\star - \mu_a$
$N_a(T)$	number of pulls of arm a up to time T
$\hat{\mu}_{a,n}$	sample mean of arm a after n pulls
R_T	expected cumulative regret
$\mathcal{G}$	good event where all empirical means stay in their confidence bands

Appendix B. Full Experiment Script

Listing 7.1: Full experiment script.

```
1
2 import numpy as np
3 import pandas as pd
4 import matplotlib.pyplot as plt
5
6 def run_once(policy, probs, horizon, rng, epsilon=0.10):
7     K = len(probs)
8     counts = np.zeros(K, dtype=int)
9     sums = np.zeros(K, dtype=float)
10    alpha = np.ones(K)
11    beta = np.ones(K)
12    mu_star = np.max(probs)
13    cumulative_regret = np.zeros(horizon)
14
15    def pull(a):
16        r = rng.binomial(1, probs[a])
17        counts[a] += 1
18        sums[a] += r
19        alpha[a] += r
20        beta[a] += 1 - r
21        return r
22
23    regret = 0.0
24
25    # give each policy one clean round of initial data, except Thompson, which can act from
its prior.
26    if policy in ["Greedy", "Epsilon-greedy", "UCB"]:
27        for a in range(K):
28            if a >= horizon:
29                break
30            r = pull(a)
31            regret += mu_star - probs[a]
32            cumulative_regret[a] = regret
33            start = min(K, horizon)
34    else:
35        start = 0
36
37    for t in range(start, horizon):
38        if policy == "Greedy":
39            means = np.divide(sums, np.maximum(counts, 1))
40            a = int(np.argmax(means))
41        elif policy == "Epsilon-greedy":
42            if rng.random() < epsilon:
```

```

43         a = int(rng.integers(K))
44     else:
45         means = np.divide(sums, np.maximum(counts, 1))
46         a = int(np.argmax(means))
47     elif policy == "UCB":
48         means = sums / counts
49         bonus = np.sqrt(2.0 * np.log(max(t + 1, 2)) / counts)
50         a = int(np.argmax(means + bonus))
51     elif policy == "Thompson sampling":
52         theta = rng.beta(alpha, beta)
53         a = int(np.argmax(theta))
54     else:
55         raise ValueError(policy)
56
57     r = pull(a)
58     regret += mu_star - probs[a]
59     cumulative_regret[t] = regret
60
61     return cumulative_regret, counts
62
63 def main():
64     probs = np.array([0.30, 0.35, 0.42])
65     horizon = 2000
66     n_runs = 500
67     policies = ["Greedy", "Epsilon-greedy", "UCB", "Thompson sampling"]
68
69     all_regrets = {}
70     all_counts = {}
71     for policy in policies:
72         regrets = []
73         counts = []
74         for seed in range(n_runs):
75             rng = np.random.default_rng(202706 + 1009 * seed + len(policy))
76             r, c = run_once(policy, probs, horizon, rng)
77             regrets.append(r)
78             counts.append(c)
79         all_regrets[policy] = np.vstack(regrets)
80         all_counts[policy] = np.vstack(counts)
81
82     rows = []
83     for policy in policies:
84         final = all_regrets[policy][:, -1]
85         cnt = all_counts[policy]
86         rows.append({
87             "policy": policy,
88             "mean_final_regret": final.mean(),
89             "std_final_regret": final.std(ddof=1),
90             "arm0_mean_pulls": cnt[:, 0].mean(),

```

```

91         "arm1_mean_pulls": cnt[:, 1].mean(),
92         "arm2_mean_pulls": cnt[:, 2].mean(),
93     })
94     df = pd.DataFrame(rows)
95     df.to_csv("results.csv", index=False)
96
97     plt.figure(figsize=(7.2, 4.2))
98     x = np.arange(1, horizon + 1)
99     for policy in policies:
100         mean = all_regrets[policy].mean(axis=0)
101         se = all_regrets[policy].std(axis=0, ddof=1) / np.sqrt(n_runs)
102         plt.plot(x, mean, label=policy)
103         plt.fill_between(x, mean - 2 * se, mean + 2 * se, alpha=0.12)
104     plt.xlabel("round")
105     plt.ylabel("mean cumulative regret")
106     plt.title("Regret in a three-arm Bernoulli bandit")
107     plt.legend(frameon=False)
108     plt.tight_layout()
109     plt.savefig("regret_curves.pdf")
110     plt.savefig("regret_curves.png", dpi=200)
111     plt.close()
112
113     plt.figure(figsize=(7.2, 4.2))
114     labels = [r"$p=0.30$", r"$p=0.35$", r"$p=0.42$"]
115     xloc = np.arange(len(labels))
116     width = 0.18
117     for i, policy in enumerate(policies):
118         means = all_counts[policy].mean(axis=0)
119         plt.bar(xloc + (i - 1.5) * width, means, width, label=policy)
120     plt.xticks(xloc, labels)
121     plt.ylabel("mean number of pulls")
122     plt.title("Where each algorithm spends its samples")
123     plt.legend(frameon=False, fontsize=8)
124     plt.tight_layout()
125     plt.savefig("pull_counts.pdf")
126     plt.savefig("pull_counts.png", dpi=200)
127     plt.close()
128
129 if __name__ == "__main__":
130     main()

```

References

- [1] P. Auer, N. Cesa-Bianchi, and P. Fischer. “Finite-time analysis of the multiarmed bandit problem”. In: *Machine Learning* 47 (2002), pp. 235–256.
- [2] S. Bubeck and N. Cesa-Bianchi. “Regret analysis of stochastic and nonstochastic multi-armed bandit problems”. In: *Foundations and Trends in Machine Learning* 5.1 (2012), pp. 1–122.
- [3] T. Lattimore and C. Szepesvari. *Bandit Algorithms*. Cambridge University Press, 2020.
- [4] D. Russo et al. “A tutorial on Thompson sampling”. In: *Foundations and Trends in Machine Learning* 11.1 (2018), pp. 1–96.
- [5] N. Srinivas et al. “Gaussian process optimization in the bandit setting: No regret and experimental design”. In: *International Conference on Machine Learning*. 2010.